UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL



CARRERA DE SOFTWARE

Título: Implementación de un árbol binario en Java

Estudiante:

Cunalata Mendoza Damian Alexander

Modalidad:

Individual – Práctica de programación T6

Ciclo Académico y Paralelo:

Tercero “B”

Módulo y Docente:

Estructura de Datos – Ing. José Caiza.

JULIO 2026 - DICIEMBRE 2026

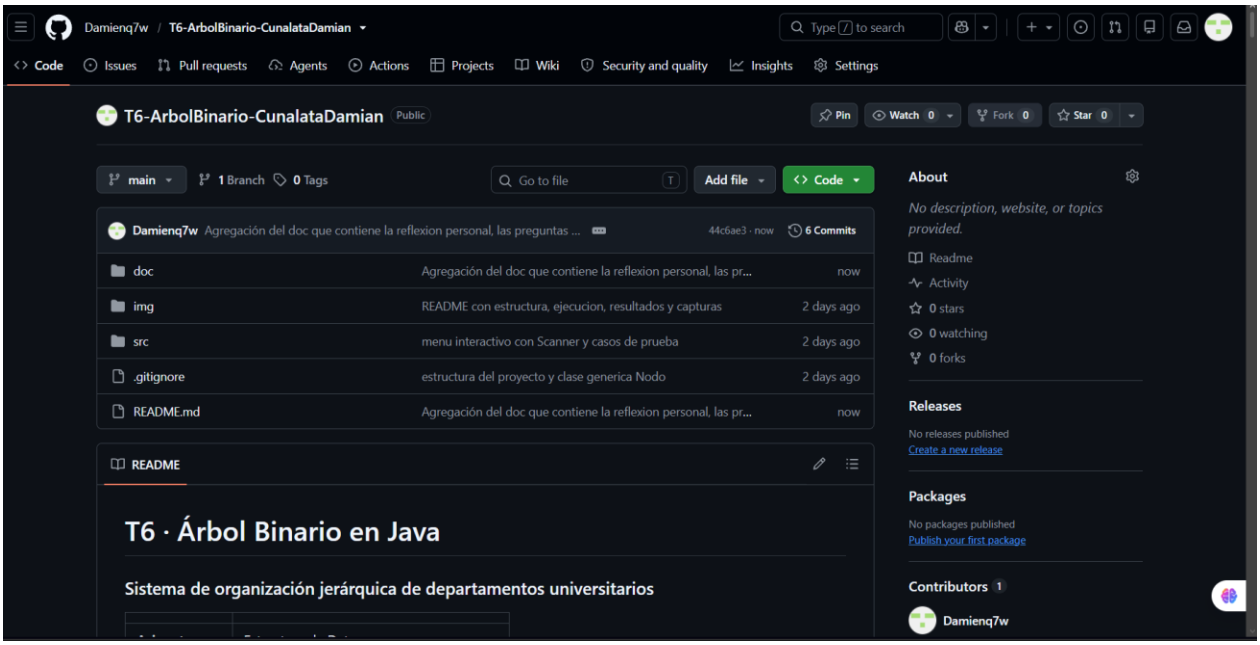
Implementación de un árbol binario en Java
Estudiante: Cunalata Mendoza Damian Alexander

En esta práctica implementé un árbol binario en Java para representar cómo se organizan las dependencias académicas de la universidad. A continuación dejo el enlace al repositorio, el historial de commits con lo que hice en cada uno, las capturas de la ejecución con los casos de prueba que pide la guía y mis respuestas a las preguntas de reflexión.

1. Enlace al repositorio

https://github.com/Damienq7w/T6-ArbolBinario-CunalataDamian

El repositorio es público y tiene el nombre que pide la guía. Adentro están las tres clases separadas en los paquetes modelo, negocio y app, el README.md con toda la documentación (problema, estructura del árbol, cómo ejecutarlo y resultados) y la carpeta img con las capturas de la ejecución.



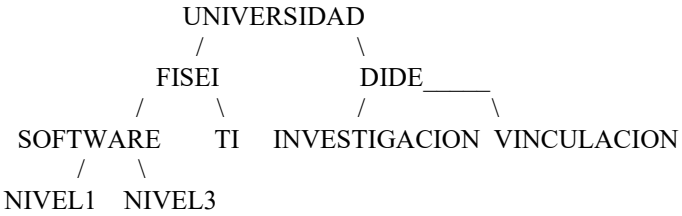
Captura de la página principal del repositorio en GitHub.

2. Estructura del proyecto y del árbol

El código está dividido en tres capas, y cada una solo usa a la que tiene debajo: Main (app) usa a ArbolBinario (negocio), y ArbolBinario usa a Nodo (modelo).

Paquete	Clase	Qué hace
modelo	Nodo<T>	Guarda el dato y los enlaces al hijo izquierdo y derecho. Tiene esHoja(), grado(), getters y setters.
negocio	ArbolBinario<T>	Guarda la raíz, valida las inserciones (una sola raíz, no pisar un hijo que ya existe) y calcula nodos, hojas y altura con recursividad. También tiene buscar() y mostrarArbol().
app	Main	Menú interactivo con Scanner, construcción del árbol inicial y casos de prueba.

El árbol que se construye es el de la guía:

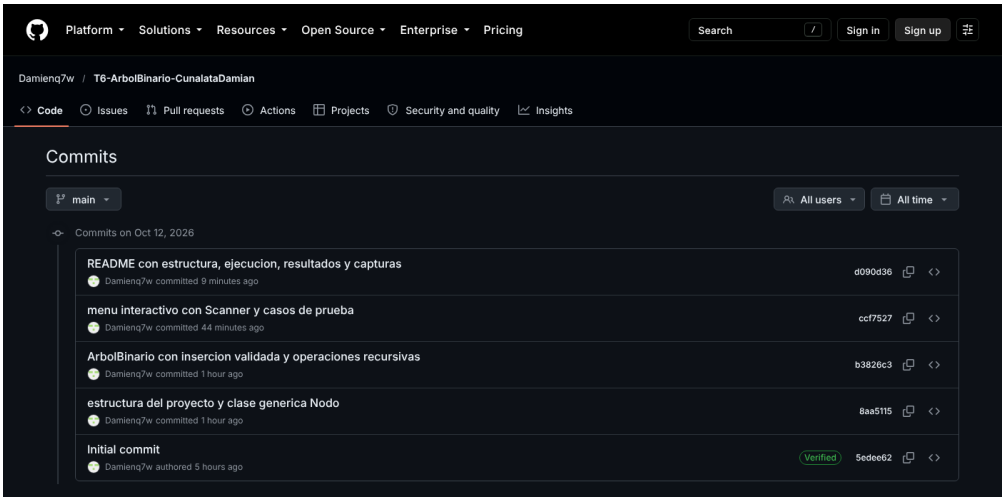


Tiene 9 nodos, 5 hojas (TI, INVESTIGACION, VINCULACION, NIVEL1 y NIVEL3) y una altura de 3 aristas, que es el camino UNIVERSIDAD → FISEI → SOFTWARE → NIVEL1.

3. Historial de commits

El desarrollo lo fui subiendo por partes, una capa a la vez, para que en el historial se vea cómo fue avanzando el proyecto:

Commit	Mensaje	Qué se hizo
8aa5115	estructura del proyecto y clase generica Nodo	Creé la estructura de paquetes (modelo, negocio, app) y la clase Nodo<T> con esHoja(), grado(), getters y setters.
b3826c3	ArbolBinario con insercion validada y operaciones recursivas	Implementé crearRaiz, agregarIzquierdo y agregarDerecho con sus validaciones, y las operaciones recursivas contarNodos, contarHojas, altura y buscar, además de mostrarArbol.
ccf7527	menu interactivo con Scanner y casos de prueba	Programé el menú, la construcción del árbol inicial, las opciones de consulta e inserción y la opción que ejecuta los casos de prueba obligatorios.
d090d36	README con estructura, ejecucion, resultados y capturas	Documenté el proyecto en el README y subí las capturas de la ejecución.



Captura del historial de commits en GitHub.

4. Capturas de ejecución

Todas las capturas son de la ejecución del programa en la terminal de Visual Studio Code. Debajo de cada una indico qué opciones ingresé y qué resultado se obtuvo.

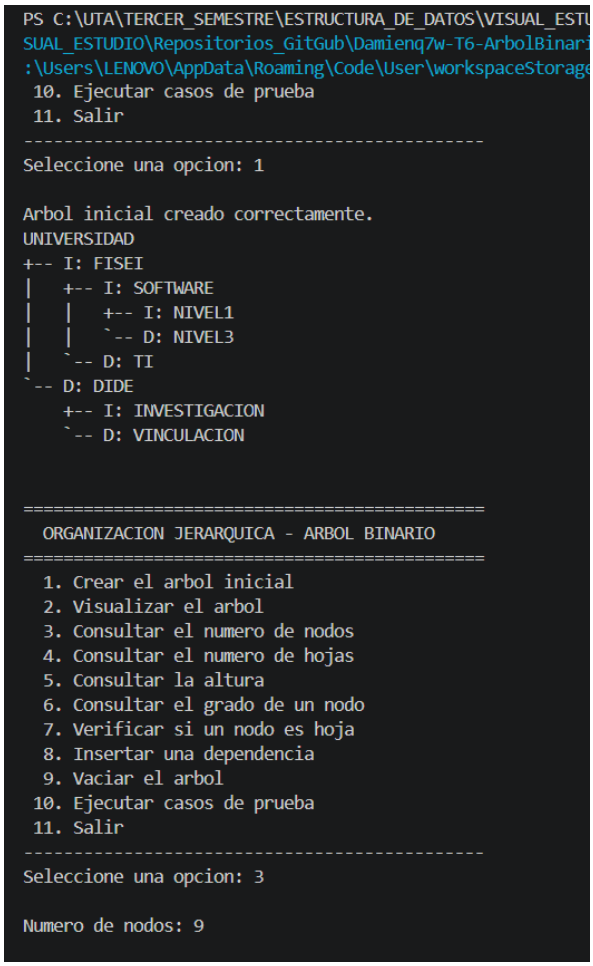
4.1 Creación del árbol inicial

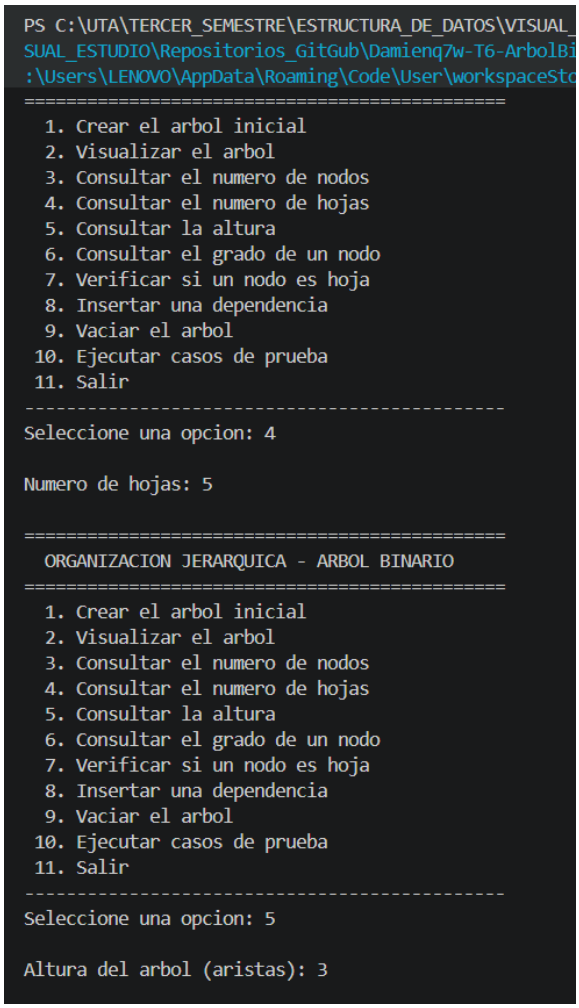
```
PS C:\UTA\TERCER_SEMESTRE\ESTRUCTURA_DE_DATOS\VISUAL_EST
SUAL_ESTUDIO\Repositorios_GitGub\Damienq7w-T6-ArbolBinar
:\Users\LENOVO\AppData\Roaming\Code\User\workspaceStorag
================================================
  1. Crear el arbol inicial
  2. Visualizar el arbol
  3. Consultar el numero de nodos
  4. Consultar el numero de hojas
  5. Consultar la altura
  6. Consultar el grado de un nodo
  7. Verificar si un nodo es hoja
  8. Insertar una dependencia
  9. Vaciar el arbol
 10. Ejecutar casos de prueba
 11. Salir
------------------------------------------------
Seleccione una opcion: 1

Arbol inicial creado correctamente.
UNIVERSIDAD
+-- I: FISEI
|   +-- I: SOFTWARE
|   |   +-- I: NIVEL1
|   |   `-- D: NIVEL3
|   `-- D: TI
`-- D: DIDE
    +-- I: INVESTIGACION
    `-- D: VINCULACION
```

Opción 1: se crea el árbol de la guía y se dibuja en consola (I = hijo izquierdo, D = hijo derecho).

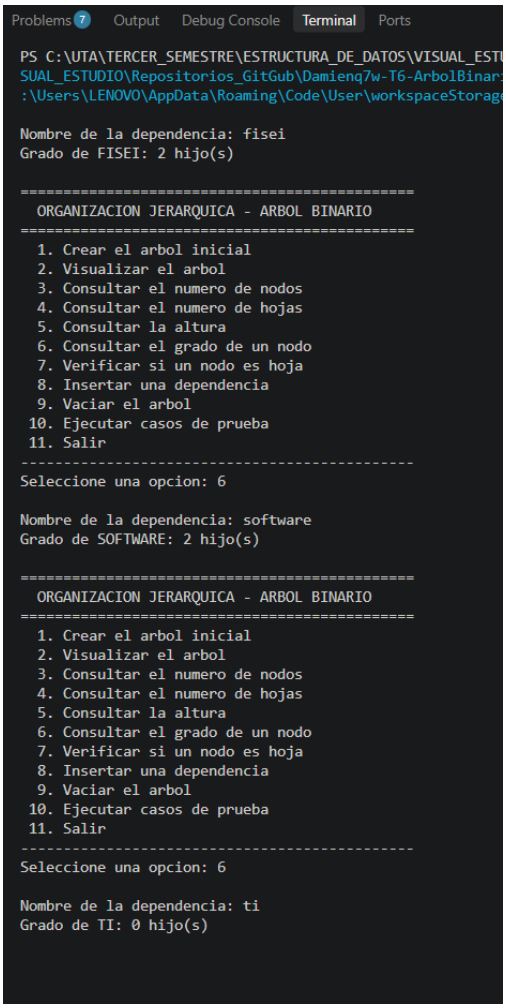
4.2 Número de nodos, hojas y altura

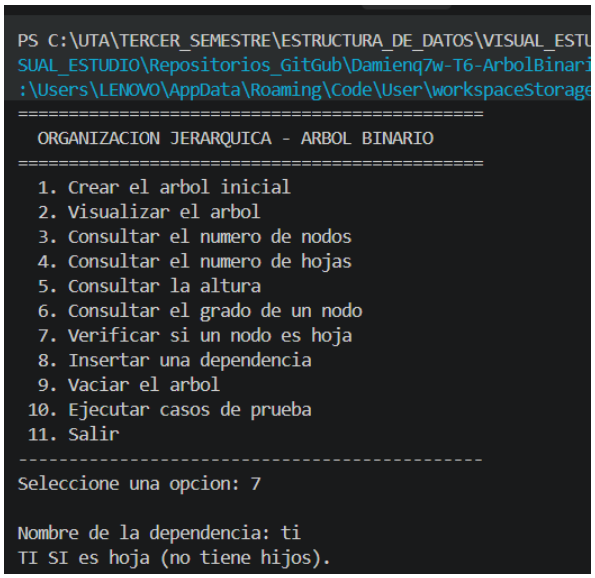




Opciones 3, 4 y 5 sobre el árbol inicial: 9 nodos, 5 hojas y altura 3.

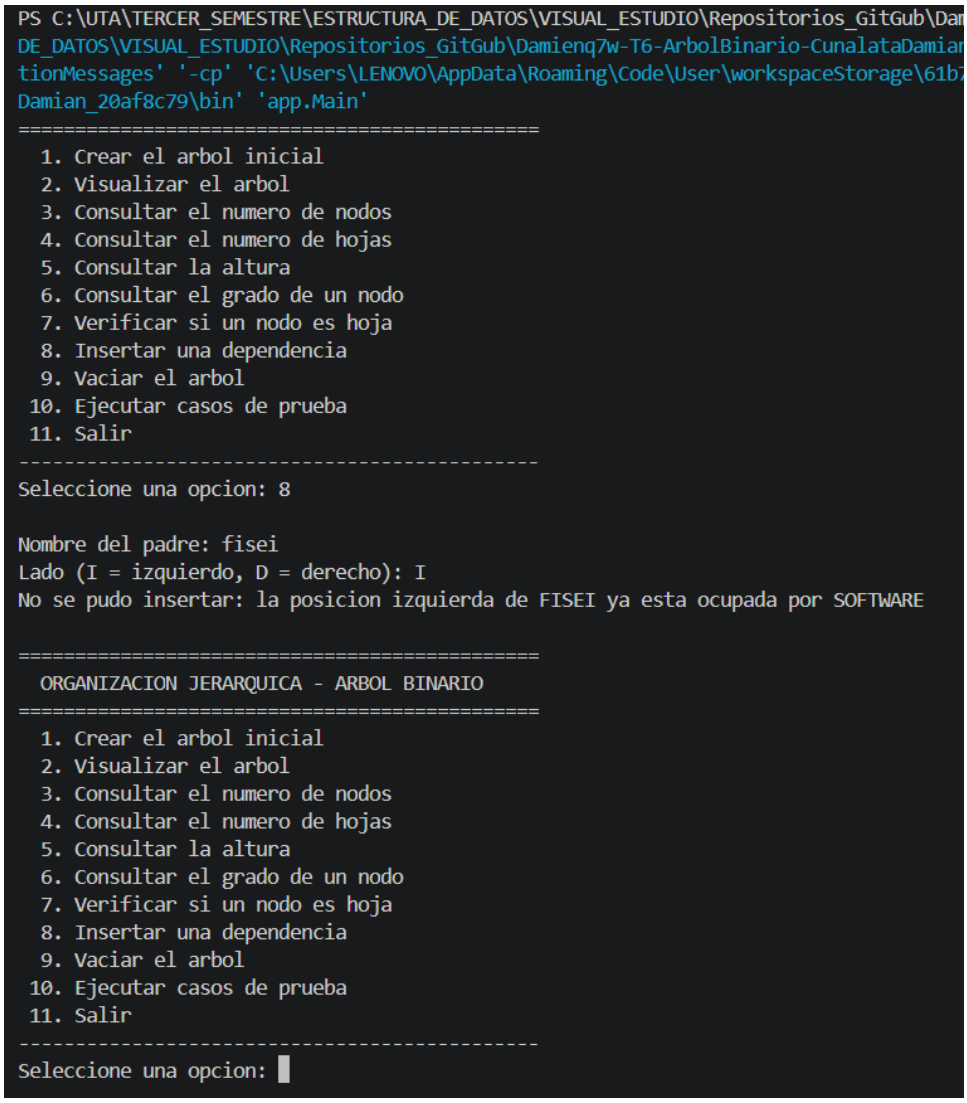
4.3 Grado de FISEI y SOFTWARE, y verificación de hoja





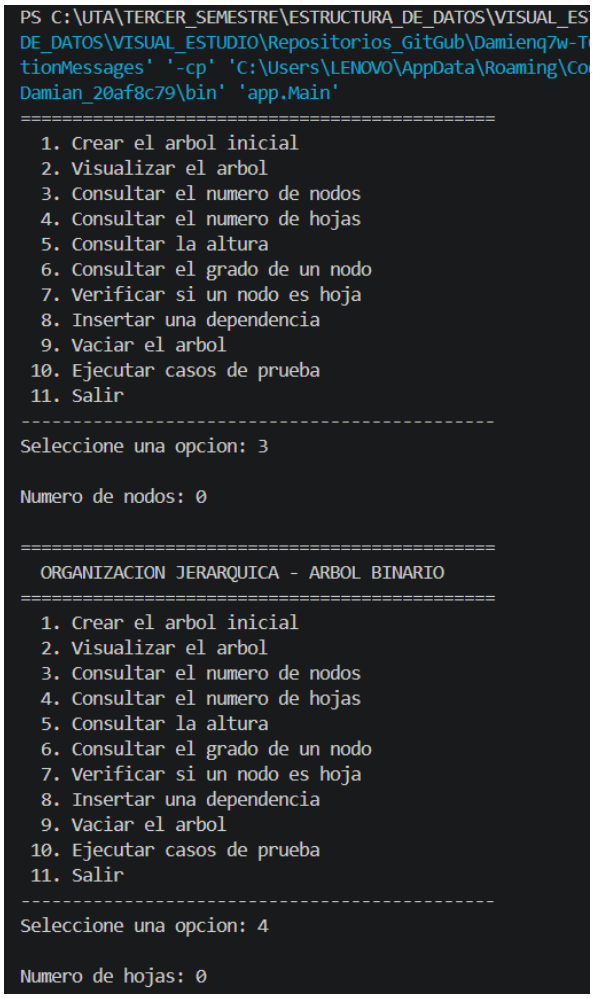
Opción 6: FISEI y SOFTWARE tienen grado 2 y TI grado 0. Opción 7: TI sí es hoja.

4.4 Intento de insertar en una posición ocupada



Opción 8: al intentar colgar un hijo a la izquierda de FISEI, el programa lo rechaza porque ese lugar ya lo ocupa SOFTWARE.

4.5 Árbol vacío

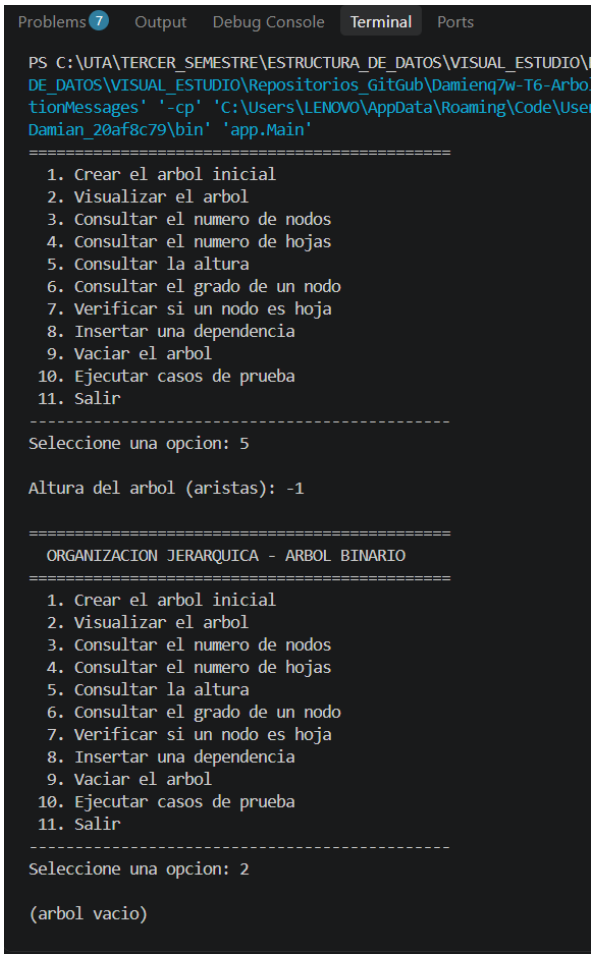


```
PS C:\UTA\TERCER_SEMESTRE\ESTRUCTURA_DE_DATOS\VISUAL_ES
DE_DATOS\VISUAL_ESTUDIO\Repositorios_GitGub\Damienq7w-T
tionMessages' '-cp' 'C:\Users\LENOVO\AppData\Roaming\Co
Damian_20af8c79\bin' 'app.Main'
==========================================
  1. Crear el arbol inicial
  2. Visualizar el arbol
  3. Consultar el numero de nodos
  4. Consultar el numero de hojas
  5. Consultar la altura
  6. Consultar el grado de un nodo
  7. Verificar si un nodo es hoja
  8. Insertar una dependencia
  9. Vaciar el arbol
 10. Ejecutar casos de prueba
 11. Salir
------------------------------------------
Seleccione una opcion: 3

Numero de nodos: 0

==========================================
  ORGANIZACION JERARQUICA - ARBOL BINARIO
==========================================
  1. Crear el arbol inicial
  2. Visualizar el arbol
  3. Consultar el numero de nodos
  4. Consultar el numero de hojas
  5. Consultar la altura
  6. Consultar el grado de un nodo
  7. Verificar si un nodo es hoja
  8. Insertar una dependencia
  9. Vaciar el arbol
 10. Ejecutar casos de prueba
 11. Salir
------------------------------------------
Seleccione una opcion: 4

Numero de hojas: 0
```



```
Problems 7  Output  Debug Console  Terminal  Ports

PS C:\UTA\TERCER_SEMESTRE\ESTRUCTURA_DE_DATOS\VISUAL_ESTUDIO\
DE_DATOS\VISUAL_ESTUDIO\Repositorios_GitGub\Damienq7w-T6-Arbo
tionMessages' '-cp' 'C:\Users\LENOVO\AppData\Roaming\Code\Use
Damian_20af8c79\bin' 'app.Main'
==========================================
  1. Crear el arbol inicial
  2. Visualizar el arbol
  3. Consultar el numero de nodos
  4. Consultar el numero de hojas
  5. Consultar la altura
  6. Consultar el grado de un nodo
  7. Verificar si un nodo es hoja
  8. Insertar una dependencia
  9. Vaciar el arbol
 10. Ejecutar casos de prueba
 11. Salir
------------------------------------------
Seleccione una opcion: 5

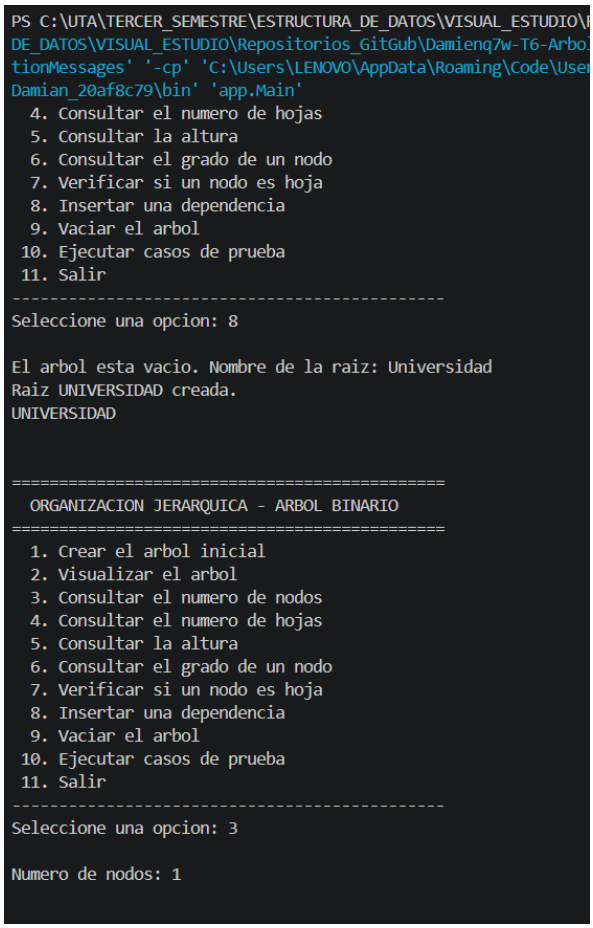
Altura del arbol (aristas): -1

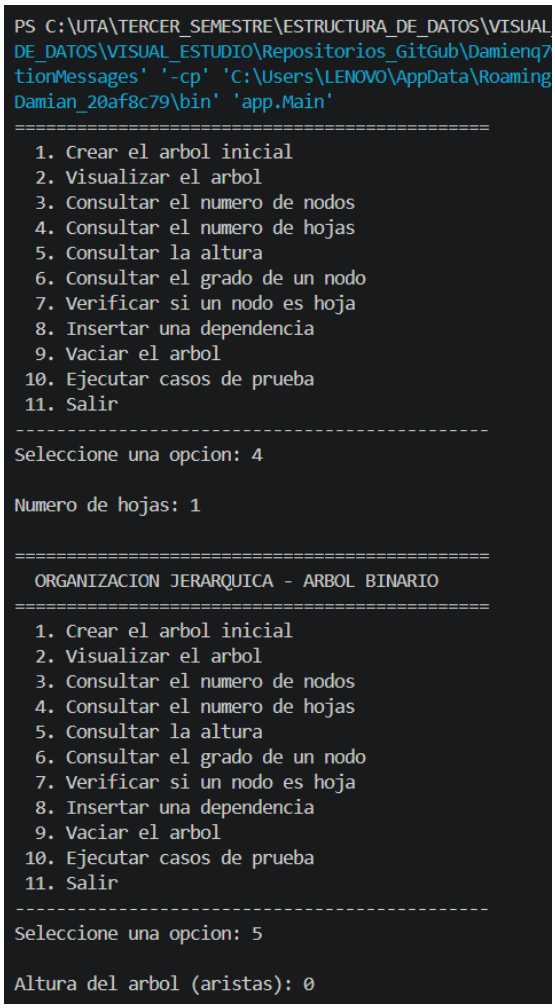
==========================================
  ORGANIZACION JERARQUICA - ARBOL BINARIO
==========================================
  1. Crear el arbol inicial
  2. Visualizar el arbol
  3. Consultar el numero de nodos
  4. Consultar el numero de hojas
  5. Consultar la altura
  6. Consultar el grado de un nodo
  7. Verificar si un nodo es hoja
  8. Insertar una dependencia
  9. Vaciar el arbol
 10. Ejecutar casos de prueba
 11. Salir
------------------------------------------
Seleccione una opcion: 2

(arbol vacio)
```

Sin crear el árbol: 0 nodos, 0 hojas, altura -1 y el dibujo muestra "(arbol vacio)".

4.6 Árbol con un solo nodo





Opción 8 con el árbol vacío: UNIVERSIDAD queda como única raíz. Resultado: 1 nodo, 1 hoja y altura 0.

4.7 Casos de prueba automáticos

```
PS C:\UTA\TERCER_SEMESTRE\ESTRUCTURA_DE_DATOS\VISUAL_ESTUDIO\Repositorios_GitGub\Da
T6-ArbolBinario-CunalataDamian'; & 'C:\Program Files\Java\jdk-21.0.11\bin\java.exe'
9aa7\redhat.java\jdt_ws\Damienq7w-T6-ArbolBinario-CunalataDamian_20af8c79\bin' 'app
------------------------------------------
Seleccione una opcion: 10

CASO 1: Arbol vacio
(arbol vacio)
  [OK] esVacio() -> esperado: true, obtenido: true
  [OK] contarNodos() -> esperado: 0, obtenido: 0
  [OK] contarHojas() -> esperado: 0, obtenido: 0
  [OK] altura() -> esperado: -1, obtenido: -1

CASO 2: Arbol con un solo nodo
UNIVERSIDAD
  [OK] contarNodos() -> esperado: 1, obtenido: 1
  [OK] contarHojas() -> esperado: 1, obtenido: 1
  [OK] altura() -> esperado: 0, obtenido: 0
  [OK] raiz.esHoja() -> esperado: true, obtenido: true

CASO 3: Arbol inicial
UNIVERSIDAD
+-- I: FISEI
|   +-- I: SOFTWARE
|   |   +-- I: NIVEL1
|   |   `-- D: NIVEL3
|   `-- D: TI
`-- D: DIDE
    +-- I: INVESTIGACION
    `-- D: VINCULACION
  [OK] contarNodos() -> esperado: 9, obtenido: 9
  [OK] contarHojas() -> esperado: 5, obtenido: 5
  [OK] altura() -> esperado: 3, obtenido: 3

CASO 4: Insertar en una posicion ocupada
  Mensaje: La posicion izquierda de FISEI ya esta ocupada por SOFTWARE
  [OK] rechaza insercion en FISEI.izquierdo -> esperado: true, obtenido: true
  Mensaje: El arbol ya tiene raiz: UNIVERSIDAD
  [OK] rechaza una segunda raiz -> esperado: true, obtenido: true
  [OK] contarNodos() sigue igual -> esperado: 9, obtenido: 9

CASO 5: Grado de FISEI y SOFTWARE
  [OK] FISEI.grado() -> esperado: 2, obtenido: 2
  [OK] SOFTWARE.grado() -> esperado: 2, obtenido: 2

RESULTADO: 16 de 16 verificaciones correctas.
```

Opción 10: se ejecutan los cinco casos obligatorios y las 16 verificaciones salen correctas.

5. Preguntas de reflexión

5.1 ¿Por qué la altura de un árbol binario depende del camino más largo desde la raíz hasta una hoja?

Porque la altura mide qué tan profundo llega el árbol y eso lo marca la rama que más baja no las demás. En un árbol no todas las ramas tienen el mismo largo, en el que se construyó, el camino UNIVERSIDAD ->DIDE -> VINCULACION tiene 2 aristas, pero UNIVERSIDAD -> FISEI -> SOFTWARE -> NIVEL1 tiene 3. Si tomara el camino más corto, el árbol parecería más pequeño de lo que es y no reflejaría el peor caso de recorrido.

Eso es justo lo que hace el método altura(), en cada nodo calcula la altura del lado izquierdo y del derecho, se queda con la mayor usando Math.max y le suma 1 por la arista que los une. Al árbol vacío se le coloca -1 para que una hoja quede en 1 + max(-1, -1) = 0, porque una hoja no tiene aristas hacia abajo. Además, la altura es importante porque dice cuántos pasos puede costar en el peor caso, llegar desde la raíz hasta un nodo.

5.2 ¿Qué diferencias existen entre almacenar información en una lista enlazada y en un árbol binario?

La diferencia principal está en los enlaces de cada nodo y en la forma que toman los datos. En una lista enlazada cada nodo tiene un solo enlace (siguiente), así que los datos quedan uno detrás de otro, en fila. En un árbol binario cada nodo tiene dos enlaces (izquierdo y derecho), y eso permite guardar jerarquía: quién depende de quién.

Aspecto	Lista enlazada	Árbol binario
Enlaces por nodo	Uno (siguiente)	Dos (izquierdo y derecho)
Forma de los datos	Lineal, en secuencia	Jerárquica, por niveles
Recorrido	De inicio a fin, en un solo sentido	Por ramas, normalmente con recursividad
Búsqueda	O(n): hay que pasar nodo por nodo	O(n) en un árbol cualquiera; O(log n) si es un BST equilibrado
Cuándo conviene	Datos en orden de llegada, colas, historiales	Organizaciones, jerarquías y búsquedas rápidas

En esta práctica se nota claramente, con una lista no habría forma de saber que SOFTWARE pertenece a FISEI y FISEI a UNIVERSIDAD. Con el árbol esa relación queda guardada en los mismos enlaces.

5.3 ¿Cómo podría utilizarse un árbol binario en el proyecto integrador SmartETL para organizar o buscar registros?

En SmartETL trabajamos con datos de condados de Estados Unidos, y por ahora los registros se guardan en una lista enlazada. El problema es que, para encontrar un condado, la lista tiene que revisarlos uno por uno, y con miles de condados y más de un millón de registros diarios eso se vuelve lento.

Un árbol binario de búsqueda (BST) usando el código FIPS del condado como clave ayudaría mucho: los códigos menores van a la izquierda y los mayores a la derecha, entonces en cada paso se descarta la mitad del árbol. Si el árbol está equilibrado, encontrar un condado entre unos 3 200 tomaría alrededor de 12 comparaciones, en vez de recorrer toda la lista. Además, si recorremos el árbol en inorden, los condados salen ordenados por su código sin tener que ordenar aparte, lo que serviría para generar los reportes de la fase Load. Habría que cuidar que el árbol no se degenere en una cadena (por ejemplo, si los códigos llegan ya ordenados), porque ahí la altura sería n - 1 y volveríamos a tener una búsqueda tan lenta como en la lista.

6. Reflexión personal

Esta práctica me ayudó a entender de verdad cómo funciona la recursividad en un árbol. Al principio me parecía raro que un método se llamara a sí mismo, pero al hacer contarNodos, contarHojas y altura me di cuenta de que siempre es la misma idea, resolver el problema para el subárbol izquierdo, luego para el derecho, y juntar los resultados, hasta llegar a null. Lo que más me costó fue entender por qué la altura de un árbol vacío es -1; tuve que hacer la traza a mano con NIVEL1, SOFTWARE y FISEI para ver que así la altura cuenta aristas y no nodos.

También aprendí la importancia de validar antes de insertar. Si no revisaba que la posición estuviera libre, el programa podía reemplazar un hijo sin avisar y se perdía toda una rama del árbol. Separar el código en modelo, negocio y app me sirvió para tener cada cosa en su lugar: las reglas están en ArbolBinario y el menú solo se encarga de hablar con el usuario.

Por último, subir el trabajo por partes a GitHub me obligó a terminar y probar cada clase antes de pasar a la siguiente. Si tuviera que hacerlo otra vez, empezaría escribiendo los casos de prueba desde el inicio para ir comprobando cada método apenas lo termino.